

Turkish Legal RAG Final Technical Report

1. Project Scope

This project implements Turkish legal Question Answering with Retrieval-Augmented Generation. It compares Base RAG and Fine-tuned RAG under the same generator LLM family and exposes a custom document and custom benchmark interface so the evaluator can run the system on a new legal document collection.

Important dataset decision: the assignment PDF lists Kaggle **turkishlaw-dataset-for-llm-finetuning** and Hugging Face **Renicames/turkish-lawchatbot** as possible finetune corpora, but this project uses only **Datasets_Ceng493_legal_rag** as requested by the team. The two external assignment datasets were not used for training or evaluation.

2. Assignment Requirement Mapping

Requirement	Status	Evidence
-------------	--------	----------

|---|---:|---|

Turkish legal RAG architecture	Done	src/legal_rag/pipeline.py
Base RAG vs Fine-tuned RAG with same LLM	Done	scripts/evaluate.py, scripts/gpu_rag_evaluate.py
Gold QA benchmark with 150-300 questions	Done	240 rows in gold_benchmark.json
Retrieval metrics	Done	Recall@k, MRR@10, nDCG@k
QA metrics	Done	EM, token F1, ROUGE-L
Citation and faithfulness metrics	Done	citation hit, top1 citation accuracy, token support
Hallucination analysis	Done	unsupported sentence rate and error analysis
Ablation study	Done	base, retriever, reranker, LLM, full
GPU fine-tuning	Done	artifacts/gpu/*/training_summary.json
Custom document support	Done	--custom-docs
Custom benchmark support	Done	--benchmark-file

3. Dataset Preparation

The dataset directory contains 7,579 corpus rows, 2,059 embedding training rows, 6,752 reranker training rows, 13,758 LLM instruction rows, a 240-question gold benchmark, and a 1,000-row RAG evaluation set. Corpus rows are loaded from **corpus.jsonl**. The benchmark stores **question**, **verified_answer**, and **gold_sources**; **source_id** and **corpus_row_id** are used as relevant document identifiers.

Preparation steps:

- Corpus rows are indexed with **id**, **title**, **text**, and metadata such as category and citation label.
- Embedding rows are converted into query, positive passage, and hard negative triples.
- Reranker rows are converted into query, candidate passage, and binary label pairs.
- LLM rows are converted into instruction-style source-grounded answer examples.
- Benchmark rows are kept separate from training and are used only for evaluation.

4. Selected Models and Rationale

Component	Model	Why selected
-----------	-------	--------------

|---|---|---|

Generator LLM	Qwen/Qwen2.5-3B-Instruct	Turkish-capable instruction model, feasible on 8GB GPU with QLoRA
Embedding	intfloat/multilingual-e5-base	Strong multilingual retrieval baseline with query/passage prefix format
Reranker	BAAI/bge-reranker-v2-m3	Multilingual cross-encoder suited for query-document relevance

Base RAG and Fine-tuned RAG use the same generator model family. Fine-tuned RAG attaches a LoRA adapter to the same Qwen base model instead of changing the LLM.

5. Finetuning Strategy

- Embedding model: contrastive triplet training on query, positive passage, and hard negative passage examples from **embedding.jsonl**.
- Reranker: cross-encoder binary classification on query-candidate pairs from **reranker.jsonl**.
- LLM: QLoRA/LoRA supervised fine-tuning on **llm.jsonl**, using source-grounded legal answer instructions.
- RAG prompting: the generator is instructed to answer only from retrieved legal sources and include citations.

GPU training was executed on NVIDIA GeForce RTX 4060 8GB.

Component	Steps	Average loss	Output
-----------	-------	--------------	--------

|---|---|---|

Embedding	300	0.0561	artifacts/gpu/embedding_model
Reranker	30	0.3735	artifacts/gpu/reranker_model
LLM LoRA	60	0.9744	artifacts/gpu/llm_lora

6. System Architecture

The pipeline is:

Question -> Retriever -> Candidate Documents -> Optional Reranker -> Context Builder -> Same LLM -> Answer with Citations

Two implementations are provided:

- Lightweight reproducible pipeline: pure Python BM25-style retrieval and local ablation weights. This is fast and used for the full 240-question benchmark.
- GPU neural pipeline: dense E5 retrieval, BGE reranking, and Qwen generation with optional LoRA adapter. This validates the actual fine-tuned model artifacts.

7. Evaluation Metrics

Retrieval metrics: Recall@1, Recall@3, Recall@5, Recall@10, MRR@10, nDCG@5, and nDCG@10.

Answer metrics: Exact Match, token F1, and ROUGE-L. Exact Match is strict and expected to be low because Turkish legal answers are abstractive and include citations.

Grounding metrics: citation hit rate, top-1 citation accuracy, grounded answer rate, faithfulness token support, and unsupported sentence rate. Unsupported sentence rate is used as a hallucination proxy.

8. Full Gold Benchmark Results

The full 240-question gold benchmark was run with **scripts/evaluate.py**.

System	R@1	R@5	R@10	MRR@10	nDCG@10	F1	ROUGE-L	Citation hit	Faithfulness
--------	-----	-----	------	--------	---------	----	---------	--------------	--------------

|---|---|---|---|---|---|---|---|

Base	0.875	0.971	0.979	0.913	0.929	0.395	0.370	0.954	0.730
Fine-tuned	0.879	0.971	0.979	0.914	0.930	0.398	0.373	0.946	0.728

Interpretation: on the full lightweight benchmark, the fine-tuned pipeline gives a small gain in Recall@1, MRR@10, nDCG, F1, and ROUGE-L. Citation hit rate is slightly lower but remains high, so the improvement should be reported as modest rather than overstated.

9. Ablation Study

The ablation variants are implemented in **scripts/evaluate.py**:

- Base: baseline retrieval, baseline reranking, baseline answer style.
- Adapted retriever: only retrieval weights are adapted.
- Adapted reranker: only reranker weights are adapted.
- Adapted LLM: only answer style and instruction behavior are adapted.
- Fine-tuned: adapted retriever, adapted reranker, and adapted answer style together.

This directly addresses the requirement to show individual component contributions. The GPU neural script also compares base dense embedding plus base LLM against fine-tuned embedding plus reranker plus LoRA adapter.

10. GPU Neural Validation Results

GPU retrieval-only validation on the first 20 gold benchmark questions:

System	R@1	R@5	R@10	MRR@10	nDCG@10	Citation hit
--------	-----	-----	------	--------	---------	--------------

|---|---|---|---|---|---|

Base neural RAG	0.60	0.85	0.85	0.67	0.713	0.90
Fine-tuned neural RAG	0.85	0.85	0.85	0.85	0.850	0.90

GPU generation smoke on one gold question:

System	R@1	F1	ROUGE-L	Citation hit	Faithfulness
--------	-----	----	---------	--------------	--------------

|---|---|---|---|---|

Base neural RAG	0.00	0.250	0.219	0.00	0.375
Fine-tuned neural RAG	1.00	0.870	0.870	1.00	0.690

This small generation test is not treated as the main statistical result, but it confirms that the LoRA path loads on GPU and can generate source-grounded legal answers.

11. Embedding-Only Neural Results

Embedding-only checks show mixed behavior:

Benchmark	System	R@1	R@5	R@10	MRR@10
-----------	--------	-----	-----	------	--------

|---|---|---|---|---|

Gold first 100	Base E5	0.620	0.830	0.880	0.708
Gold first 100	Fine-tuned E5	0.520	0.700	0.740	0.604
RAG eval first 200	Base E5	0.865	0.970	0.990	0.915

Gold first 100	Base E5	0.620	0.830	0.880	0.708
RAG eval first 200	Fine-tuned E5	0.855	0.970	0.995	0.905

This means embedding fine-tuning alone is not uniformly better. The best neural improvement appears when fine-tuned embedding is combined with reranking and LoRA generation. The report therefore does not claim that every component independently improves every metric.

12. Custom Data and Benchmark Support

The evaluator can provide a custom document collection and benchmark without changing code:

```
``powershell .\conda-envs\legal-rag-gpu\python.exe scripts\evaluate.py --custom-docs path\to\docs.jsonl
--benchmark-file path\to\benchmark.json --artifacts artifacts --out reports\teacher_eval.json ``
```

Custom document formats:

- Directory of **.txt** files.
- Single **.txt** file.
- **.jsonl** with **id**, **text**, optional **title**, optional **metadata.citation_label**.
- **.json** list of the same objects.

Custom benchmark format:

```
``json [ { "question": "Question text", "verified_answer": "Gold answer", "gold_sources": [{"source_id":
"doc_id", "corpus_row_id": "doc_id", "citation_label": "Citation label"}] } ] ``
```

The project includes **examples/custom_docs.jsonl** and **examples/custom_benchmark.json**. The smoke test reached Recall@1 = 1.0, Citation hit = 1.0, and Top-1 citation accuracy = 1.0 on this custom input.

13. Hallucination and Error Analysis

Observed failure modes:

- Base neural RAG can retrieve a nearby legal article and then answer as if the target article is absent. This happened in the generation smoke test for CMK m.225 when the top document was CMK m.218.
- Exact Match is near zero because generated answers include citations and paraphrasing; token F1 and ROUGE-L better reflect answer quality.
- Some lightweight answers concatenate too much context when high-overlap neighboring articles are retrieved. This increases token support but can lower concise answer quality.
- Fine-tuned embedding alone can underperform the base E5 model on some gold subsets, likely because the limited training schedule and hard-negative distribution overfit some patterns.

Mitigations:

- Evaluate Recall@1 and top-1 citation accuracy, not only Recall@10.
- Require citations in answers.
- Measure unsupported sentence rate as a hallucination proxy.
- Provide reranking and LoRA generation as separate ablations.

Recommended next improvements:

- Train the embedding model longer with validation-based checkpoint selection.
- Cache dense document embeddings for faster full neural evaluation.
- Add human legal review for a representative sample of generated answers.

14. Reproducibility

Main commands:

```
``powershell .\conda-envs\legal-rag-gpu\python.exe scripts\train_lightweight.py --dataset  
Datasets_Ceng493_legal_rag --out artifacts .\conda-envs\legal-rag-gpu\python.exe scripts\evaluate.py  
--dataset Datasets_Ceng493_legal_rag --artifacts artifacts --benchmark gold --out  
reports\evaluation_results_gold_full_metrics.json .\conda-envs\legal-rag-gpu\python.exe  
scripts\gpu_rag_evaluate.py --dataset Datasets_Ceng493_legal_rag --benchmark gold --limit 20  
--no-generation --out reports\gpu_rag_retrieval_gold_20.json .\conda-envs\legal-rag-gpu\python.exe  
scripts\gpu_rag_evaluate.py --dataset Datasets_Ceng493_legal_rag --benchmark gold --limit 1 --out  
reports\gpu_rag_generation_gold_1.json ``
```

Verification:

```
``powershell .\conda-envs\legal-rag-gpu\python.exe -m unittest discover -s tests -p "test_*.py" ``
```

The final test run passed 3 unit tests.

15. Conclusion

The project satisfies the core assignment requirements: Turkish legal RAG, same-LLM Base vs Fine-tuned comparison, gold benchmark evaluation, ablation study, GPU fine-tuning artifacts, custom document support, custom benchmark support, and source-grounding/hallucination analysis. The current strongest claim is that the full fine-tuned pipeline modestly improves the full lightweight benchmark and clearly improves the small GPU neural generation smoke test. The report intentionally marks embedding-only neural results as mixed so the evaluation remains honest.